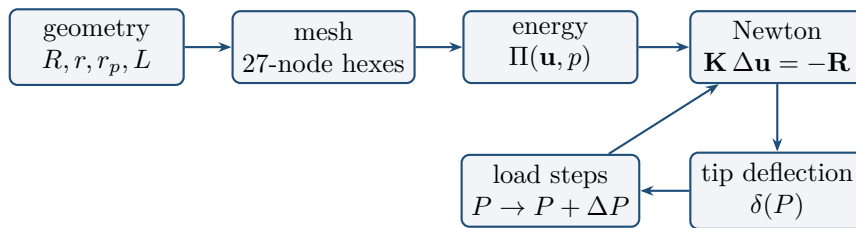


B.Tech Project

How our nonlinear 3-D FEA works

A beginner's handbook to `actuator_fea.py`



Rudranarayan Pradhan

October 2026

What this is. A step-by-step explanation of the finite-element code we wrote to predict how far the eccentric-cavity actuator bends under pressure. It assumes only first-year mechanics and calculus. Each section explains one idea in plain terms, gives the equation, and then points to the exact lines of `actuator_fea.py` that implement it, so the handbook can be read side by side with the code. Line numbers refer to the version of the code in the project folder on 8 October 2026.

Contents

1	The big picture	2
2	FEA in one dimension: the whole method on a bar	2
3	Describing large deformation	4
4	The material: hyperelastic silicone	4
5	Nearly incompressible rubber: locking and the mixed element	5
6	The 27-node element and Gauss integration	5
7	Building the actuator mesh	6
8	Supports, symmetry and the sleeve	8
9	The pressure load	8
10	Equilibrium: the total potential and the residual	9
11	Newton's method with automatic differentiation	9
12	Load stepping and the ballooning limit	10
13	Post-processing: from displacements to tip deflection	12
14	Checking the code: verification	12
15	What the runs give	13
16	Running and changing the code	14
17	Troubleshooting	15
A	Code map	15
B	Glossary	16

1 The big picture

The question. Our actuator is a silicone cylinder (radius $R = 10$ mm, length $L = 100$ mm) with a cylindrical cavity (radius $r = 4$ mm) placed $r_p = 3$ mm off the centre. When the cavity is pressurised, the thin side of the wall stretches more than the thick side and the cylinder curls towards the thick side. We want the tip deflection δ as a function of the pressure P .

Why hand formulas are not enough. The beam formulas in the derivation report assume small strains and a linear material. In our actuator the silicone stretches by up to 2.5 times, the material gets stiffer in a nonlinear way, and the pressure keeps pushing perpendicular to the wall even as the wall moves. All three effects are *nonlinear*. The finite element method (FEM, or FEA for finite element analysis) handles them by solving the full 3-D equations of large-deformation elasticity numerically.

Why we wrote our own code. No structural FE package (ANSYS, Abaqus, COMSOL) was installed. Instead we wrote a compact solver in Python (`actuator_fea.py`, about 530 lines) on top of `numpy`, `scipy` and `jax`. Writing it ourselves has one big advantage for a project: every modelling choice is visible and can be checked.

The pipeline. Figure 1 shows what happens when you run the code. Each box is a section of this handbook.

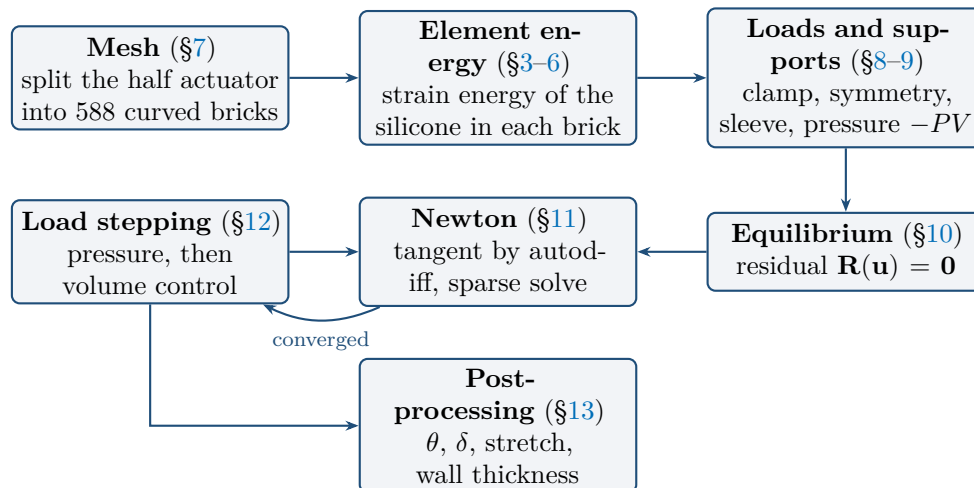


Figure 1: What `actuator_fea.py` does, in order.

Key idea FEA turns one impossible problem (find the exact deformed shape of a 3-D rubber body) into a very large but ordinary one (find about 18 000 numbers, the displacements of the mesh nodes, that make the forces balance).

2 FEA in one dimension: the whole method on a bar

Everything the 3-D code does already appears in a 1-D bar, so we start there.

Step 1: cut the body into elements. Take a bar of length $3a$, axial stiffness EA , clamped at the left and pulled by a force F at the right. Cut it into three *elements* joined at four *nodes* (Figure 2).

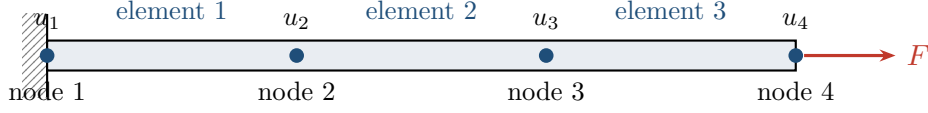


Figure 2: A bar cut into three two-node elements.

Step 2: guess a shape inside each element. Inside an element we do not know the displacement $u(X)$, so we *interpolate* it from the node values with *shape functions*. For a two-node element of length a the simplest choice is a straight line:

$$u(\xi) = N_1(\xi) u_i + N_2(\xi) u_j, \quad N_1 = \frac{1}{2}(1 - \xi), \quad N_2 = \frac{1}{2}(1 + \xi), \quad (2.1)$$

where $\xi \in [-1, 1]$ is a local coordinate along the element. Our 3-D code uses quadratic instead of linear shape functions, in three directions (§6), but the idea is identical.

Step 3: write the energy. The strain in an element is $\varepsilon = (u_j - u_i)/a$ and its stored energy is $\frac{1}{2}EAa\varepsilon^2 = \frac{1}{2}k(u_j - u_i)^2$ with $k = EA/a$. The total potential energy is the stored energy minus the work done by the load:

$$\Pi(\mathbf{u}) = \frac{1}{2}k(u_2 - u_1)^2 + \frac{1}{2}k(u_3 - u_2)^2 + \frac{1}{2}k(u_4 - u_3)^2 - F u_4. \quad (2.2)$$

Step 4: equilibrium means the energy is stationary. The bar settles where Π is a minimum, $\partial\Pi/\partial u_n = 0$ for every free node. Writing this out gives the familiar matrix equation $\mathbf{K}\mathbf{u} = \mathbf{f}$:

$$k \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 1 \end{bmatrix} \begin{bmatrix} u_2 \\ u_3 \\ u_4 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ F \end{bmatrix} \implies u_2 = \frac{F}{k}, \quad u_3 = \frac{2F}{k}, \quad u_4 = \frac{3F}{k} = \frac{F \cdot 3a}{EA}. \quad (2.3)$$

$u_1 = 0$ is the clamp: its row and column are simply removed. Each element contributes a small 2×2 block $k \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$ to the big matrix; adding these blocks into the right rows and columns is called *assembly*.

Step 5: what changes when the problem is nonlinear. For a rubber bar the force is no longer proportional to the stretch. For an incompressible neo-Hookean bar the nominal stress is $s = \mu(\lambda - \lambda^{-2})$, so to find the stretch at which $s = 0.5\mu$ we must solve $R(\lambda) = \lambda - \lambda^{-2} - 0.5 = 0$. We cannot rearrange it, so we use *Newton's method*: linearise, solve, repeat,

$$\lambda_{k+1} = \lambda_k - \frac{R(\lambda_k)}{R'(\lambda_k)}, \quad R'(\lambda) = 1 + 2\lambda^{-3}. \quad (2.4)$$

iteration k	λ_k	residual $R(\lambda_k)$
0	1.000000	-5.0×10^{-1}
1	1.166667	-6.8×10^{-2}
2	1.196774	-1.4×10^{-3}
3	1.197429	-6.3×10^{-7}
4	1.197429	-1.2×10^{-13}

The number of correct digits roughly doubles every iteration: this is *quadratic convergence*, and it is exactly what we see in the 3-D runs, where each pressure step converges in 3 to 6 iterations.

The whole method in four words **Interpolate, integrate, assemble, iterate.** The 3-D code does these same steps with 27-node bricks instead of 2-node bars, a rubber energy instead of $\frac{1}{2}k\varepsilon^2$, and about 18 000 unknowns instead of three.

3 Describing large deformation

Reference and current positions. Every material point has a position $\mathbf{X}$ in the undeformed (*reference*) body and a position $\mathbf{x} = \mathbf{X} + \mathbf{u}$ after deformation, where $\mathbf{u}$ is the displacement. All our unknowns are nodal displacements.

The deformation gradient. How a tiny fibre $d\mathbf{X}$ is stretched and rotated is described by the 3×3 matrix

$$\mathbf{F} = \frac{\partial \mathbf{x}}{\partial \mathbf{X}}, \quad d\mathbf{x} = \mathbf{F} d\mathbf{X}. \quad (3.1)$$

Two quantities built from $\mathbf{F}$ matter for rubber:

- the **volume ratio** $J = \det \mathbf{F}$ ($J = 1$ means no volume change);
- the **shape-change invariant** $\bar{I}_1 = J^{-2/3} \text{tr}(\mathbf{F}^\top \mathbf{F}) = J^{-2/3} \sum_{ij} F_{ij}^2$, which is 3 in the undeformed state and grows as the material is distorted.

$\bar{I}_1$ ignores volume change (the factor $J^{-2/3}$ removes it), so it separates “changing shape” from “changing volume”. This split is what lets us treat incompressibility separately (§5).

Stretch. The principal stretches $\lambda_1 \geq \lambda_2 \geq \lambda_3$ are the square roots of the eigenvalues of $\mathbf{C} = \mathbf{F}^\top \mathbf{F}$. The largest one, λ_1 , is what we plot as “peak principal stretch”.

In the code `_elem_pi` (line 184) computes $\mathbf{F}$ at all 27 Gauss points from the current nodal positions, `J = _det3(F)` (line 178), and `d = J**(-2/3) * sum(F*F) - 3`, which is $\bar{I}_1 - 3$. `FEModel.stretch` (line 303) computes λ_1 per element with `np.linalg.eigvalsh`.

4 The material: hyperelastic silicone

Rubber-like materials are *hyperelastic*: all their behaviour comes from a *strain-energy density* W , the energy stored per unit reference volume. We use the Yeoh model:

$$W_{\text{iso}} = C_{10}(\bar{I}_1 - 3) + C_{20}(\bar{I}_1 - 3)^2 + C_{30}(\bar{I}_1 - 3)^3. \quad (4.1)$$

With $C_{20} = C_{30} = 0$ this is the **neo-Hookean** model, which we use for the placeholder silicone. The constants link to the familiar small-strain moduli by

$$\mu = 2C_{10} \quad (\text{shear modulus}), \quad E = 3\mu = 6C_{10} \quad (\text{incompressible}). \quad (4.2)$$

For $E = 0.6$ MPa this gives $C_{10} = 0.1$ MPa and $\mu = 0.2$ MPa. The higher Yeoh terms make the silicone stiffen at large stretch; their values must come from a tensile test of the real silicone (a task in the project timeline).

In the code `Material` in `actuator_model.py` stores (C_{10}, C_{20}, C_{30}) ; `Material.from_E(E)` sets $C_{10} = E/6$. In `_elem_pi` the three Yeoh terms are `c[0]*d + c[1]*d**2 + c[2]*d**3`. The command-line flag `--yeoh C10 C20 C30` replaces the neo-Hookean default.

5 Nearly incompressible rubber: locking and the mixed element

The problem. Silicone hardly changes volume: its Poisson's ratio is almost 0.5. We model this with a large bulk modulus κ ,

$$\kappa = \frac{2\mu(1+\nu)}{3(1-2\nu)}, \quad \nu = 0.4995 \Rightarrow \kappa \approx 1000\mu. \quad (5.1)$$

If we simply added a stiff volume penalty $\frac{1}{2}\kappa(J-1)^2$ to W , the elements would *lock*: with only a few displacement modes per element, keeping $J \approx 1$ at every Gauss point is so restrictive that the mesh becomes far too stiff and the predicted bending far too small. Locking is the classic trap of rubber FEA.

The fix: a separate pressure field. We add a second unknown, an internal pressure p , and use the *mixed* energy

$$W = W_{\text{iso}}(\bar{I}_1) + p(J-1) - \frac{p^2}{2\kappa}. \quad (5.2)$$

Making this stationary with respect to p gives $J-1 = p/\kappa$: the volume constraint is now enforced through p rather than through the displacements. Inside each element p is linear in the local coordinates,

$$p(\xi, \eta, \zeta) = p_0 + p_1\xi + p_2\eta + p_3\zeta, \quad (5.3)$$

so it has only four coefficients and is allowed to jump between elements. This is the well-known **Q2/P1** element (quadratic displacement, linear discontinuous pressure), which does not lock.

Static condensation: getting rid of p . Because p belongs to one element only, it can be eliminated element by element before assembly. Linearising one element gives the block system

$$\begin{bmatrix} \mathbf{K}_{uu} & \mathbf{K}_{up} \\ \mathbf{K}_{pu} & \mathbf{K}_{pp} \end{bmatrix} \begin{bmatrix} \Delta \mathbf{u} \\ \Delta p \end{bmatrix} = - \begin{bmatrix} \mathbf{r}_u \\ \mathbf{r}_p \end{bmatrix}, \quad \mathbf{K}_{pp} = -\frac{1}{\kappa} \mathbf{M}, \quad \mathbf{M} = \int \boldsymbol{\psi} \boldsymbol{\psi}^T dV, \quad (5.4)$$

where $\boldsymbol{\psi} = (1, \xi, \eta, \zeta)$ and $\mathbf{M}$ is the small “pressure mass matrix”. With $\mathbf{A} = -\mathbf{K}_{pp}^{-1} = \kappa \mathbf{M}^{-1}$ the second row gives

$$\Delta p = \mathbf{A} (\mathbf{r}_p + \mathbf{K}_{pu} \Delta \mathbf{u}), \quad (5.5)$$

and substituting into the first row leaves a displacement-only system,

$$(\mathbf{K}_{uu} + \mathbf{K}_{up} \mathbf{A} \mathbf{K}_{pu}) \Delta \mathbf{u} = -(\mathbf{r}_u + \mathbf{K}_{up} \mathbf{A} \mathbf{r}_p). \quad (5.6)$$

The global matrix therefore has only displacement unknowns, and the pressures are recovered afterwards from the formula for Δp .

In the code `FEModel.__init__`: `self.kappa` (line 253) and `self.A` (line 261). `PSI` (line 59) holds $(1, \xi, \eta, \zeta)$ at the Gauss points. In `FEModel.solve`: the condensed tangent `H_uu + KA @ Kup.T` (line 334), the extra residual `Rc = KA @ rp` (line 340), and the pressure update `dp` (line 351).

6 The 27-node element and Gauss integration

Quadratic shape functions. In one direction we use three nodes at $\xi = -1, 0, 1$ and the quadratic Lagrange functions

$$N_{-1} = \frac{1}{2}\xi(\xi-1), \quad N_0 = 1 - \xi^2, \quad N_{+1} = \frac{1}{2}\xi(\xi+1). \quad (6.1)$$

Each equals 1 at its own node and 0 at the other two. Multiplying one function from each of the three directions gives $3 \times 3 \times 3 = 27$ shape functions for a brick with 27 nodes: 8 corners, 12 edge midpoints, 6 face centres and 1 centre.

The isoparametric map. The same shape functions describe the element's geometry: the reference cube $[-1, 1]^3$ is mapped onto the real, curved brick by $\mathbf{X}(\boldsymbol{\xi}) = \sum_a N_a(\boldsymbol{\xi}) \mathbf{X}_a$ (Figure 3). Because the functions are quadratic, element edges can be curved, which lets a coarse mesh follow the circular cavity and outer surface closely. The chain rule then gives the derivatives we need for $\mathbf{F}$:

$$\frac{\partial N_a}{\partial \mathbf{X}} = \frac{\partial N_a}{\partial \boldsymbol{\xi}} \mathbf{J}_0^{-1}, \quad \mathbf{J}_0 = \frac{\partial \mathbf{X}}{\partial \boldsymbol{\xi}}. \quad (6.2)$$

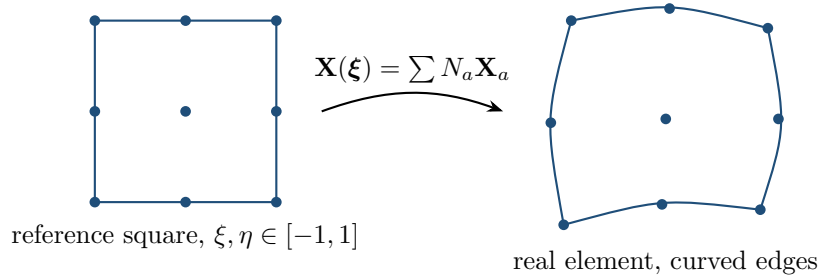


Figure 3: The isoparametric idea, drawn in 2-D (a 9-node face of the 27-node brick).

Gauss integration. The element energy $\int W dV$ cannot be integrated by hand, so we evaluate W at a few special points and take a weighted sum. Three Gauss points per direction, at $\xi = -\sqrt{0.6}, 0, +\sqrt{0.6}$ with weights $5/9, 8/9, 5/9$, integrate polynomials up to degree five exactly. In 3-D that is 27 points, and

$$\int_{\text{element}} W dV \approx \sum_{g=1}^{27} w_g \det \mathbf{J}_0(\boldsymbol{\xi}_g) W(\mathbf{F}_g). \quad (6.3)$$

In the code `_lagrange` (line 41) gives N and $dN/d\xi$; `_GP`, `_GW` (lines 49–50) are the Gauss points and weights; `_tp3` (line 54) builds the 27 tensor products; `DN3` and `W3` (lines 57–58) are their derivatives and weights. `FEModel.__init__` stores `dNdX` (line 258) and `wdV = w_g \det \mathbf{J}_0` (line 259) once, since the reference mesh never changes.

7 Building the actuator mesh

Half model. The actuator and its loading are symmetric about the bending plane $z = 0$, so we mesh only the half $z \geq 0$ and impose symmetry on the cut (§8). This halves the cost with no loss of accuracy.

The cross-section: rays from the cavity centre. The wall is meshed with rays from the cavity centre $C = (0, r_p)$ at angle ϕ from the thin side. Each ray starts at the cavity (distance r) and ends on the outer circle, at distance

$$\rho_{\text{out}}(\phi) = \sqrt{R^2 - r_p^2 \sin^2 \phi} - r_p \cos \phi = r + t(\phi), \quad (7.1)$$

which is exactly the wall-thickness formula $t(\phi)$ from the derivation report. Nodes are spaced evenly along each ray. The section is then copied along x to make the 3-D wall (Figure 4).

The tip cap. At $x = L$ the cavity is closed by a solid cap of thickness $R/2 = 5$ mm. Filling the circular cavity area with bricks needs a small square core and a ring of elements around it, so the cap is built from three blocks.

Gluing blocks together. The blocks are built separately and then merged: nodes that sit at the same point are found with a k -d tree and given one number. Nodes are numbered along x , which keeps the stiffness matrix narrow (banded) and the sparse solver fast.

Mesh size. The “medium” mesh used for all results has 12 elements round the half circumference, 2 through the wall and 20 along the length, plus the cap: 588 elements, 6205 nodes and 17 725 free displacement unknowns. Going to the fine mesh (1664 elements) changes the tip deflection by less than 0.5 % (§14).

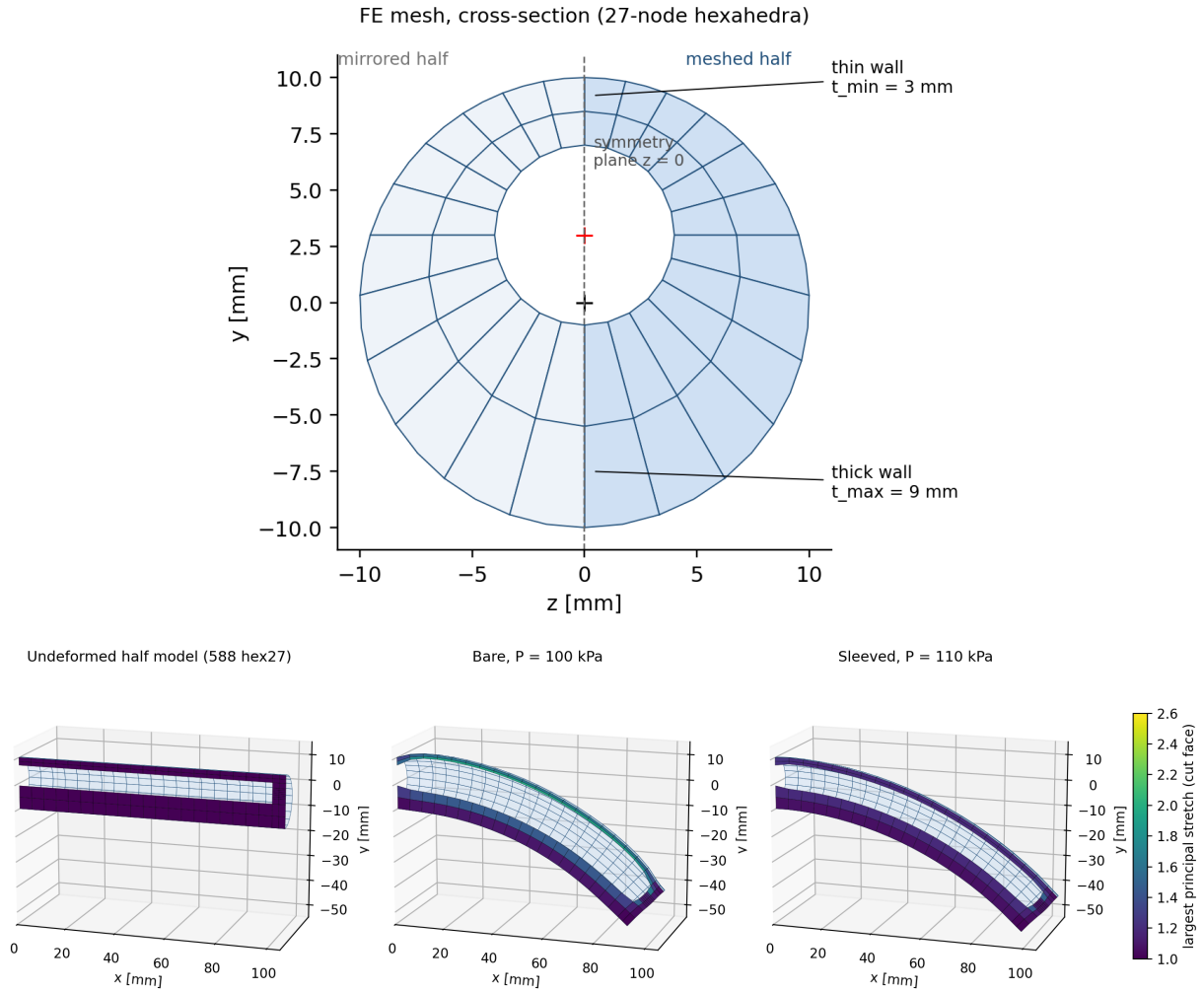


Figure 4: Top: the cross-section mesh, built from rays out of the cavity centre. Bottom: the 3-D half model, undeformed and deformed.

In the code MESH_PRESETS (line 68), build_mesh (line 131): the ray formula is line 143, the cap blocks lines 147–155, the merge line 158. merge_blocks (line 87) uses cKDTree; right_handed (line 106) flips any element whose local axes came out left-handed (negative Jacobian); orient_faces (line 114) makes every cavity face normal point out of the fluid.

8 Supports, symmetry and the sleeve

Clamp. The base face $x = 0$ is glued to a fixed mount: all three displacement components of its nodes are zero.

Symmetry. On the cut plane $z = 0$ points can move in x and y but not through the plane: $u_z = 0$. This is the mathematical statement that the missing half mirrors the meshed half.

The sleeve (reinforced actuator only). A hoop-reinforced actuator has inextensible fibres wound round the outside. We model them as an extra energy on the outer surface that resists stretching in the hoop direction only:

$$\Pi_{\text{sleeve}} = k_s \int_{\text{outer}} \frac{1}{8} (\lambda_h^2 - 1)^2 dA, \quad k_s = 100 E R. \quad (8.1)$$

For small strain $\lambda_h = 1 + \varepsilon_h$ this is $\frac{1}{2} k_s \varepsilon_h^2$ per unit area, a hoop membrane of stiffness k_s . A membrane of radius R under pressure P carries tension PR , so its hoop strain is about $PR/k_s = P/(100E)$, small enough to act as nearly inextensible while keeping the equations well conditioned.

In the code `make_actuator` (line 388) sets the clamp (`fixed[m["base"]] = True`), the symmetry condition (`fixed[m["sym"], 2] = True`) and `sleeve_k = 100*E*R` (line 396). `_sleeve_energy` (line 203) is the hoop-fibre energy of one outer face. Fixed unknowns are removed from the system through `self.free` (line 275).

9 The pressure load

A follower load. Air pressure always pushes perpendicular to the wall, wherever the wall has moved to. Such a load *follows* the deformation, which makes it harder to handle than a fixed force. The neat way to include it is through energy: a gas at pressure P does work $P \Delta V$ when the cavity volume grows, so its contribution to the total potential is

$$\Pi_{\text{load}} = -P V(\mathbf{x}). \quad (9.1)$$

Differentiating with respect to the nodal positions automatically gives forces normal to the *deformed* cavity surface, with the right magnitude.

Computing the cavity volume. By the divergence theorem, any vector field with divergence 1 can turn a volume into a surface integral. We choose the field $\frac{1}{2}(0, y, z)$:

$$V = \frac{1}{2} \oint (y n_y + z n_z) da. \quad (9.2)$$

This choice is deliberate. On the open base ($x = 0$) the normal points along x , so $n_y = n_z = 0$; on the symmetry plane $z = 0$ and $n_y = 0$. Both contribute nothing, so we only need to integrate over the wetted faces: the cavity wall and the inside of the cap. The check at the start of every run prints the computed volume next to the exact $\pi r^2 L = 5026.5 \text{ mm}^3$.

Load stiffness. Because the load depends on the deformed shape, it also changes the tangent matrix: the term $-P \partial^2 V / \partial \mathbf{x}^2$. Leaving it out would still give the correct answer but would slow Newton's method from quadratic to linear convergence.

In the code `_face_volume` (line 194) integrates $\frac{1}{2}(yn_y + zn_z)$ over one 9-node cavity face. `FEModel.volume` (line 289) sums it; `forces` (line 292) returns its gradient $\mathbf{gV} = \partial V / \partial \mathbf{x}$; the load stiffness is `-P * self._h_vol(...)` (line 339).

10 Equilibrium: the total potential and the residual

Putting the pieces together, the total potential energy of the half actuator is

$$\Pi(\mathbf{u}, p) = \sum_{\text{elements}} \int \left[W_{\text{iso}}(\bar{I}_1) + p(J - 1) - \frac{p^2}{2\kappa} \right] dV + \Pi_{\text{sleeve}} - P V(\mathbf{u}). \quad (10.1)$$

The deformed shape is the one where Π is stationary. Differentiating with respect to every free nodal displacement gives the **residual**, the out-of-balance force at each node:

$$\mathbf{R}(\mathbf{u}) = \underbrace{\frac{\partial}{\partial \mathbf{u}} \left(\sum \int W dV + \Pi_{\text{sleeve}} \right)}_{\text{internal forces } \mathbf{f}} - P \underbrace{\frac{\partial V}{\partial \mathbf{u}}}_{\mathbf{g}} = \mathbf{0}. \quad (10.2)$$

In words: at equilibrium the silicone's internal forces exactly balance the pressure forces at every node. Together with $\partial \Pi / \partial p = 0$ (the volume constraint) this is the full set of equations.

In the code The residual is one line, `R = (f - P * gV)` (line 318). `f` and `gV` come from `FEModel.forces` (line 292), which also returns `rp = \partial \Pi / \partial p` for each element.

11 Newton's method with automatic differentiation

The iteration. Exactly as for the 1-D rubber bar, we linearise the residual about the current guess and solve for a correction:

$$\mathbf{R}(\mathbf{u} + \Delta \mathbf{u}) \approx \mathbf{R}(\mathbf{u}) + \mathbf{K} \Delta \mathbf{u} = \mathbf{0} \implies \mathbf{K} \Delta \mathbf{u} = -\mathbf{R}, \quad \mathbf{K} = \frac{\partial \mathbf{R}}{\partial \mathbf{u}}. \quad (11.1)$$

K is the **tangent stiffness matrix**: the second derivative (Hessian) of the energy. Then $\mathbf{u} \leftarrow \mathbf{u} + \Delta \mathbf{u}$ and we repeat until **R** is tiny.

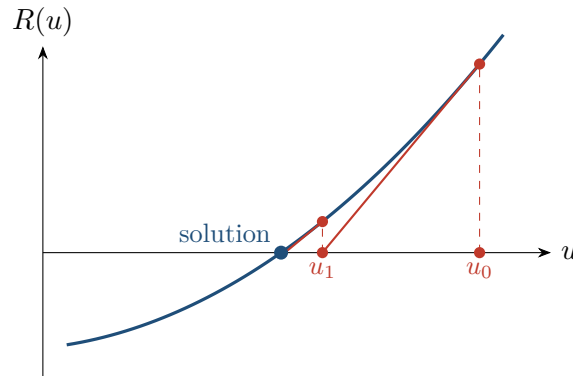


Figure 5: One Newton step: follow the tangent from the current guess to where it crosses zero. Repeating from u_1 converges quadratically.

Automatic differentiation: no hand-derived stiffness. Deriving $\mathbf{K}$ by hand for a mixed, hyperelastic, follower-load element is long and error-prone. We avoid it entirely. We only write the *energy* of one element as a Python function of its 81 nodal coordinates and 4 pressure coefficients, and the `jax` library differentiates that function exactly (not by finite differences):

- `jax.grad(_elem_pi)` gives the element force vector (85 entries);
- `jax.hessian(_elem_pi)` gives the element stiffness matrix (85×85);
- `jax.vmap` applies them to all 588 elements at once, and `jax.jit` compiles them to fast machine code the first time they are called.

The same is done for the cavity volume and the sleeve energy.

Assembly and the linear solve. The condensed element matrices ((5.6)) are added into one sparse global matrix of size 17725×17725 . Its sparsity pattern never changes, so the index bookkeeping is done once. The linear system is solved with a sparse direct LU factorisation (SuperLU).

When is it converged? A step is accepted when all three hold:

1. the relative force imbalance $\|\mathbf{R}\|/\|\mathbf{f}\| < 10^{-9}$;
2. the incompressibility constraint $J - 1 = p/\kappa$ is met to 10^{-9} in every element;
3. under volume control (§12), the cavity volume matches its target.

The iteration gives up after 15 iterations, if a result becomes NaN, or if the residual has not halved over the last four iterations.

Safeguard: do not turn elements inside out. A full Newton step can be too long and push an element to $J \leq 0$ (turned inside out), where $J^{-2/3}$ is undefined and the forces become NaN. The code then halves the step, $\alpha = 1, \frac{1}{2}, \frac{1}{4}, \dots$, until the forces are finite again. This is a simple *line search*.

In the code `FEModel.solve` (line 309) is the Newton loop. The `jax` derivatives are set up in lines 268–273; `_Assembler` (line 211) does the assembly; `spla.splu` (line 341) factorises; the convergence test is line 326; the step-halving line search is lines 353–361.

Why 64-bit numbers `jax` uses 32-bit floats by default, which are not accurate enough for a residual tolerance of 10^{-9} . Line 36 switches on 64-bit arithmetic. Do not remove it.

12 Load stepping and the ballooning limit

Why steps. Newton's method only converges from a guess close to the answer. Jumping straight from $P = 0$ to $P = 100$ kPa would fail, so the pressure is raised in steps (22 steps of 5 kPa for the runs in the deck) and each converged state is the starting guess for the next.

A better starting guess. Instead of starting the next step from the last state, the code extrapolates along the straight line through the last two states (a *secant predictor*). This usually saves one or two Newton iterations per step.

Cutting the step. If a step fails, it is halved and retried, up to twice. A step is also rejected if the cavity volume jumps by more than $0.25V_0$, which would mean Newton has leapt to a distant, snapped-through shape instead of following the path.

The limit point. For the bare actuator the pressure–volume curve rises, flattens and reaches a peak at $P \approx 104.7$ kPa (Figure 6). Beyond the peak there is *no* equilibrium at a higher pressure: in a real test the thin wall would balloon suddenly. At the peak the tangent matrix becomes singular, and pressure steps can no longer converge.

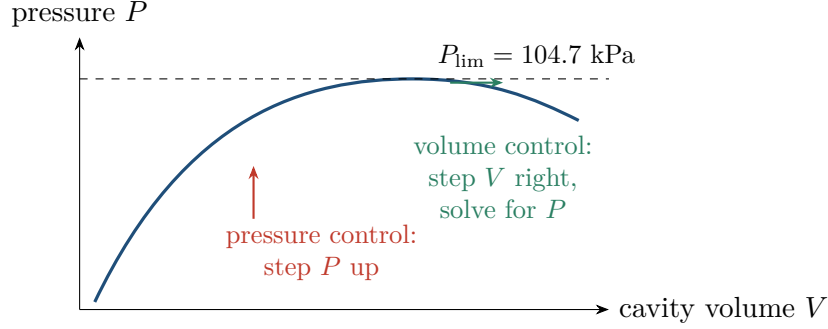


Figure 6: Pressure control cannot pass the peak of the P – V curve; volume control can. The real curve from the FE run is in Figure 7.

Volume control. The cure is to swap roles: prescribe the cavity volume (as a syringe would) and let the pressure be the unknown. One extra equation, $V(\mathbf{u}) = V_{\text{target}}$, is added. Linearising both equations,

$$\mathbf{K} \Delta \mathbf{u} - \mathbf{g} \Delta P = -\mathbf{R}, \quad \mathbf{g}^T \Delta \mathbf{u} = -(V - V_{\text{target}}), \quad (12.1)$$

with $\mathbf{g} = \partial V / \partial \mathbf{u}$. Solving the first for $\Delta \mathbf{u} = \Delta \mathbf{u}_R + \Delta P \mathbf{b}$, with $\mathbf{K} \Delta \mathbf{u}_R = -\mathbf{R}$ and $\mathbf{K} \mathbf{b} = \mathbf{g}$, and inserting into the second gives

$$\Delta P = -\frac{(V - V_{\text{target}}) + \mathbf{g}^T \Delta \mathbf{u}_R}{\mathbf{g}^T \mathbf{b}}. \quad (12.2)$$

Both solves reuse the same LU factorisation, so volume control costs almost nothing extra. This “bordered” system stays solvable at the peak, so the path can be followed over the top and the peak pressure read off as the limit pressure.

Switching rule. The code ramps under pressure control. When a pressure step still fails after two halvings, it switches to volume control with volume steps of $0.02V_0$, enlarged by 1.5 after easy steps (at most $0.1V_0$) and halved after failures. It stops when the pressure starts to fall (the peak has been passed), when the last requested pressure is reached, or when the volume exceeds $6V_0$.

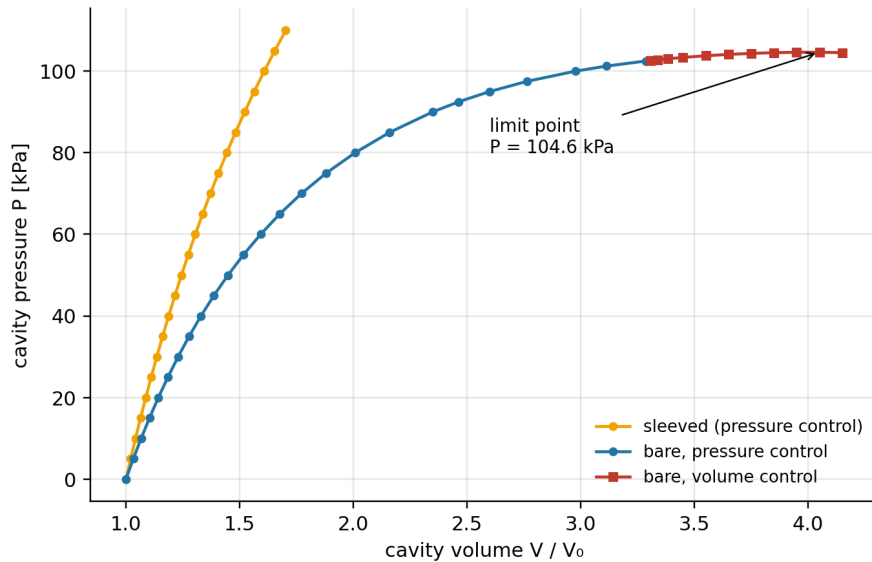


Figure 7: The FE pressure–volume path. The sleeved actuator (orange) stays on the rising branch; the bare actuator (blue) reaches its limit and is followed past it under volume control (red).

In the code `simulate` (line 399): the `predictor` (line 414), the pressure loop with step halving (lines 426–447) and the snap-through check (line 434), the volume-control loop (lines 449–463), and the limit pressure `P_limit` (line 472). The bordered solve is lines 344–348 of `FEModel.solve`.

13 Post-processing: from displacements to tip deflection

The solver returns the displacement of every node. The quantities we report are extracted from two lines of nodes on the symmetry plane: the top line of the outer surface ($y = +R$, thin side) and the bottom line ($y = -R$, thick side).

- At each x , the vector from the bottom to the top node gives the rotation of the cross-section, $\theta = \text{atan2}(d_x, d_y)$; its midpoint is the beam axis.
- The **tip angle** and **tip deflection** are θ and $\delta = -c_y$ at $x = L$, the end of the pressurised length (positive towards the thick wall).
- The **curvature** is the slope of θ against x over the middle half of the actuator, away from the clamp and the cap.
- Also recorded: axial stretch of the axis, diameter and cavity stretch at mid-span, thin-wall thickness relative to its original 3 mm, cavity volume and the largest principal stretch.

In the code `measure` (line 370). `save` (line 479) writes `summary.csv` (one row per pressure), `results.npz` (all fields) and `run.json` (inputs); `fea_viewer.py` turns them into an interactive viewer.html.

14 Checking the code: verification

A new FE code must be tested on problems whose answers are known before its predictions are trusted. `fea_verify.py` runs four such tests (Figure 8).

Test	What it checks	Worst error
1 Uniaxial tension	material law and large stretch; exact stress formula, λ up to 3	0.2 % (neo-Hookean)
2 Thick-tube inflation	follower pressure, volume formula; exact neo-Hookean solution	0.05 %
3 Small-pressure bending	the $(1 - 2\nu)$ law of the derivation report, ν from 0 to 0.4995	0.0002
4 Mesh convergence	medium vs fine mesh, tip deflection at $P/\mu = 0.30$ and 0.45	0.5 %

The Yeoh case in test 1 differs by up to 2.5 % at $\lambda = 3$. That is expected, not a bug: the exact formula assumes perfect incompressibility, while the code uses a finite κ ($\nu = 0.4995$), and the difference grows with the stiffening Yeoh terms at large stretch.

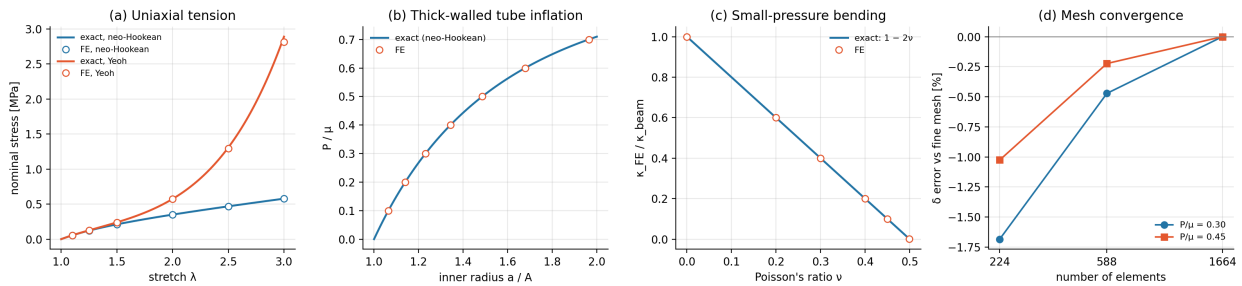


Figure 8: The four verification tests (from `results/verification.log`).

15 What the runs give

Two pressure sweeps were run on the medium mesh with the placeholder geometry and silicone: the bare actuator (36 converged states, 177 s) and the sleeved actuator to 110 kPa (23 states, 103 s). The full discussion is in the derivation report and the slides. In short:

- the bare actuator barely bends at low pressure (δ grows as P^2), then balloons and reaches its limit at 104.7 kPa;
- the sleeved actuator bends smoothly and nearly linearly, 16.3 mm at 60 kPa against 5.9 mm for the bare one;
- the beam models of the derivation track the sleeved FE within 5 % at low pressure and fall 20–25 % short by 60 kPa.

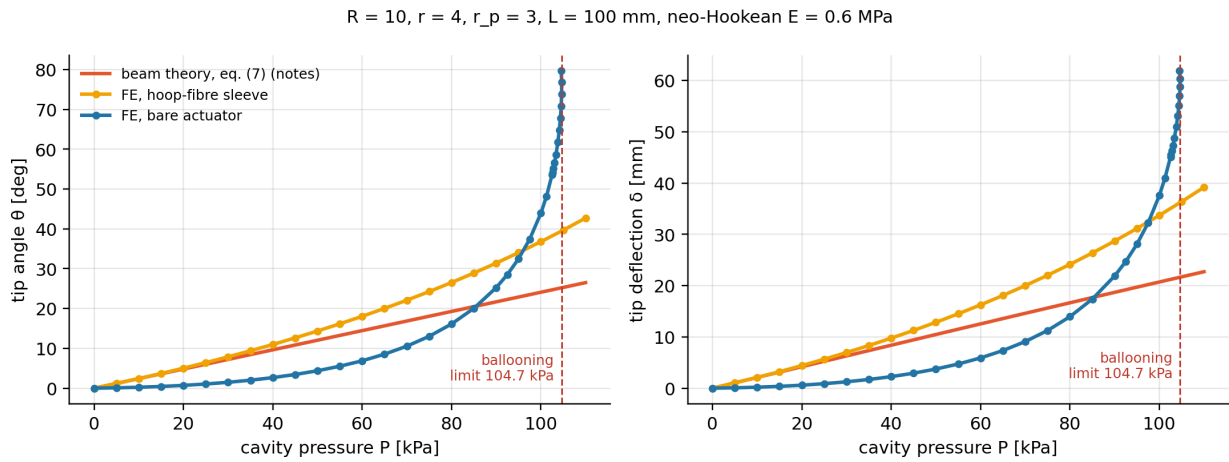


Figure 9: Tip angle and tip deflection against pressure: beam theory, bare and sleeved FE.

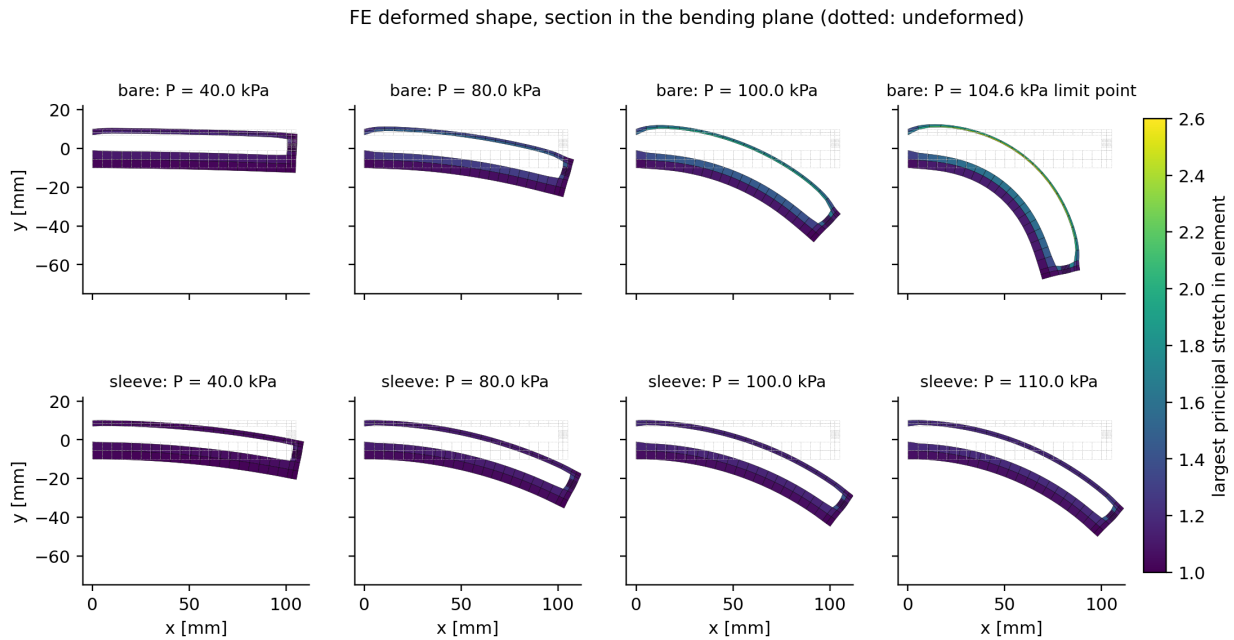


Figure 10: Deformed bending-plane sections, coloured by the largest principal stretch.

16 Running and changing the code

A pressure sweep. From the project folder:

```
python actuator_fea.py --R 10 --r 4 --rp 3 --L 100 --E 0.6 --pmax 110 --steps 22 --out
  results/bare
python actuator_fea.py --R 10 --r 4 --rp 3 --L 100 --E 0.6 --pmax 110 --steps 22
  --sleeve --out results/sleeve
python -u fea_verify.py
python fea_compare.py results/bare results/sleeve
```

Each sweep prints one line per converged state (pressure, volume ratio, tip deflection, Newton iterations, elapsed time) and ends by writing `summary.csv` and an interactive `viewer.html`.

What to change, and where.

To change	Do this
geometry	--R --r --rp --L, and --tcap for the tip cap
silicone (measured)	--yeoh C10 C20 C30 from the tensile-test fit
reinforcement	add --sleeve
pressure range	--pmax (kPa) and --steps
accuracy vs speed	--mesh coarse, medium or fine
compressibility	--nu (keep it at 0.4995 for silicone)

17 Troubleshooting

“inverted element in the mesh”

The geometry is invalid, usually $r + r_p \geq R$ (the cavity cuts through the outer wall). Check the inputs.

A pressure step fails repeatedly

Use more steps (smaller ΔP). If it happens near the top of the range on a bare actuator, it is the ballooning limit, and the switch to volume control is the correct behaviour.

The run is slow

The first call compiles the `jax` functions (tens of seconds). After that a medium sweep takes 2–3 minutes on a laptop. The fine mesh takes roughly five times longer.

Results look too stiff

Check that ν was not set much closer to 0.5 and that the mixed formulation was not removed; both would bring back locking.

Long runs vanish

Write the log to a file with `python -u ... > run.log` and watch the file; very long background jobs may be stopped by the terminal.

A Code map

Name in <code>actuator_fea.py</code>	Line	What it does
<code>_lagrange, _GP, _GW</code>	41–50	1-D quadratic shape functions, Gauss points and weights
<code>_tp3, DN3, W3, PSI</code>	54–59	27-node brick: derivatives, weights, pressure basis
<code>MESH_PRESETS</code>	68	coarse / medium / fine mesh sizes
<code>merge_blocks</code>	87	glue mesh blocks, number nodes along x
<code>right_handed, orient_faces</code>	106, 114	fix element and face orientation
<code>build_mesh</code>	131	half-actuator mesh from rays $\rho_{\text{out}}(\phi)$ and the cap
<code>_elem_pi</code>	184	mixed Yeoh energy of one element
<code>_face_volume</code>	194	cavity volume from one face
<code>_sleeve_energy</code>	203	hoop-fibre energy of one outer face
<code>_Assembler</code>	211	sparse global matrix with a fixed pattern
<code>FEModel.__init__</code>	245	κ , $\mathbf{A}$, <code>dNdX</code> , <code>jax</code> derivatives, free unknowns
<code>FEModel.forces</code>	292	internal forces, $\partial V / \partial \mathbf{u}$, constraint residuals
<code>FEModel.stretch</code>	303	largest principal stretch per element
<code>FEModel.solve</code>	309	Newton loop, condensation, volume control, line search
<code>measure</code>	370	tip angle, deflection, curvature, stretches, wall thickness
<code>make_actuator</code>	388	mesh, clamp, symmetry, sleeve stiffness
<code>simulate</code>	399	pressure ramp, step cutting, switch to volume control
<code>save</code>	479	write <code>results.npz</code> , <code>summary.csv</code> , <code>run.json</code>
command line	506	options and defaults

B Glossary

Assembly Adding each element's small matrix into the right rows and columns of the global matrix.

Automatic differentiation Computing exact derivatives of a program by applying the chain rule to every operation (here, `jax`).

Deformation gradient $\mathbf{F}$ The matrix that maps small fibres from the undeformed to the deformed body.

Follower load A load whose direction follows the deforming surface, such as gas pressure.

Gauss point A point inside an element where the integrand is evaluated for numerical integration.

Hyperelastic A material whose stress follows from a stored-energy function W .

Isoparametric Using the same shape functions for the geometry and the displacement.

Limit point A peak of the load–response curve; beyond it the load cannot increase.

Locking Artificial over-stiffness of displacement-only elements for nearly incompressible materials.

Mixed (u–p) element An element with displacement and pressure unknowns, which avoids locking.

Newton–Raphson Iteration that repeatedly solves the linearised equations; converges quadratically near the solution.

Residual The out-of-balance nodal force; zero at equilibrium.

Shape function The interpolation function attached to one node of an element.

Static condensation Eliminating element-internal unknowns before assembly.

Tangent stiffness The derivative of the residual with respect to the displacements.

Volume control Prescribing the cavity volume and solving for the pressure, used to pass a limit point.